

Project 4: Choice Topic

AE 623, Computational Fluid Dynamics II, Winter 2026

Proposal due March 27, 11:59pm, on Canvas

Class presentations: April 13, 15, 20

Code and report due: April 24, 11:59pm, on Canvas

1 Problem Description

In this project you will work with your group on a topic of your choosing. The topic should be relevant to the material covered in the course. Some project ideas are given here, but you are not restricted to these topics. The work can be related to your research or another class, as long as novel aspects pertinent to 623 are clearly stated and are addressed to completion within the time frame of this project. The discretization need not be discontinuous Galerkin, but it can be and you are allowed to use your previous project codes as a starting point.

2 Project Ideas

1. **Implicit Solvers:** Explicit solvers are cheap per iteration but may require many iterations to converge. Implicit solvers are more expensive per iteration but allow for larger time steps, which is especially useful for steady-state calculations. This project entails a comparison of implicit and explicit solvers for steady/unsteady simulations of the Euler equations. The key part is the implementation of a linearization of the residual, a process that can be accelerated through automatic differentiation. Note, once you have a linearization, you can compute adjoints.
2. **Adjoint-based Error Estimation:** Adjoint-based output error estimates quantify the effect of discretization errors on scalar outputs of interest. They rely on adjoint solutions, which are obtained from an equation that involves the residual Jacobian matrix – i.e. a linearization of the residual, as required in an implicit method. The goal of this project is to implement and study the effectivity of an output error estimate for a high-order CFD discretization.
3. **Adjoint-based Sensitivities:** Similar to the above project, another use of adjoints is to compute sensitivities of outputs to inputs or geometry. This information can then be used to drive gradient-based optimization.
4. **Mesh Adaptation:** A mesh can be adapted by adding nodes/elements in areas that are deemed important based on a solution error indicator – and possibly taking nodes/elements out where the solution error is low. Adapting an unstructured mesh can be performed using local operators, including node insertion, edge swapping, edge collapse, node movement, etc. This project involves implementing adaptive mesh capability (more sophisticated than in your previous projects) in a CFD solver and testing it using various indicators. The indicators can include simple solution features (e.g. gradients of key variables), or something more sophisticated such as residuals or output-based error indicators.
5. **ALE:** Using an arbitrary Lagrangian-Eulerian (ALE) approach, mesh/geometry motion can be modeled. This project consists of implementing such capability into a flow solver, running test cases for a static geometry (but moving mesh), and demonstrating the approach for a test case with moving geometry (e.g. a pitching or plunging airfoil).
6. **Diffusion:** There are various approaches for implementing diffusion in a discontinuous Galerkin discretization. A project idea is to pick one approach, apply it to the convection diffusion or

Navier-Stokes equations, and demonstrate it on a few test problems. A full Reynolds-averaged Navier-Stokes solver would be ambitious, but possible.

7. **HDG:** Hybridized/Embedded DG methods have recently been introduced as a way of reducing the number of degrees of freedom on a given mesh, without sacrificing the order of accuracy. This project involves implementing an HDG or EDG method in two dimensions, and comparing the method to standard DG in terms of accuracy and cost.
8. **3D:** Extending a flow solver to three dimensions involves new basis functions and elements, more orientation bookkeeping, different quadrature rules, and an extension of the flux and boundary conditions, among other tasks. One project idea is to implement such an extension and demonstrate it for a three-dimensional problem. Starting from an extruded two-dimensional geometry might help in debugging.
9. **Parallelization:** Accelerating a solver by parallelization can have a tremendous impact on the speed of execution. This project involves parallelizing your flow solver, e.g. using either MPI or OpenMP, and performing a scaling study for various orders and mesh sizes. The implementation could also include porting your code to GPUs.
10. **Other Equations:** Applying DG to equations other than Euler, or adding new physics such as turbulence models or chemical reactions.
11. **Shocks:** Implementing a limiter in your high-order DG code and running cases with shocks.
12. **Flux Reconstruction:** Implementing an alternate high-order unstructured discretization, such as flux reconstruction.
13. **Optimization:** Using your code to solve an optimization or inverse problem (e.g. field inversion). This will require adjoint capability for efficiency.
14. **Scale-Resolving Simulations:** With an efficient high-order explicit discretization and enough compute power, you can tackle large-eddy simulations (LES). To begin, you can consider “implicit” LES, which does not use a sub-grid scale (SGS) model, but instead relies on the dissipation inherent to the discretization to model the unresolved scales.
15. **Data-Driven Methods:** Closures for turbulence models can be corrected through field-inversion and machine-learning (or other function approximations). Data can come from LES, DNS, or experiments. This project involves implementing a framework such as field-inversion with machine learning (FIML) and applying it to a turbulent flow. Note that many ingredients are required to make this project work, including RANS and adjoints.

3 Tasks and Deliverables

10% Proposal: A one-page proposal is required by **March 27th**. You will receive feedback within 1-2 days of the proposal due date. The proposal should include the following information:

- A descriptive title
- A paragraph summarizing the proposed work
- An outline of the tasks, including a rough time-line

The proposal should be submitted via Canvas in **.pdf** format.

30% Presentation: In addition to the technical report, each group must prepare slides for a 25-minute presentation, to be given to the class in the last few lectures of the course. The project

does not need to be in a finished state by the time of the presentation. The presentation should state the motivation, approach, current results, and (expected) conclusions. The slides in **.pdf** format must be uploaded to Canvas by the code/report project deadline.

60% Code and report: A archive containing your source code (**.zip** format) and a separate *concise* (**12-page maximum**) project report (**.pdf** format) are required. The report should follow the template provided for the previous projects. It should be professional, complete, and concise.

Multiple groups can work on a similar topic, but discussions among groups should be limited to high-level information exchange.